Python: exercises on dictionaries, tuples, and sets

This notebook was generated in **student** mode.

Exercise 1: Nested Dictionary Manipulation

Exercise Description

You have a nested dictionary `data = {"student1": {"math": 75, "science": 82}, "student2": {"math": 88, "science": 90}}`. Write a function `update_score(data, student, subject, new_score)` that updates the score of the specified student and subject If the student or subject doesn't exist, it should add them.

Your solution

```
def update_score(data, student, subject, new_score):
    # Write your solution here
```

```
    pass
```

Test your solution

```
# Test case 1: Update existing student's existing subject
data = {"student1": {"math": 75, "science": 82}, "student2": {"math": 88, "science":
update_score(data, "student1", "math", 95)
assert data["student1"]["math"] == 95
```

Test your solution

```
# Test case 2: Add new student with new subject
data = {"student1": {"math": 75, "science": 82}, "student2": {"math": 88, "science":
update_score(data, "student3", "history", 78)
assert "student3" in data
assert data["student3"]["history"] == 78
```

Test your solution

```
# Test case 3: Add new subject to existing student
data = {"student1": {"math": 75, "science": 82}}
update_score(data, "student1", "english", 85)
assert data["student1"]["english"] == 85
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Dictionary Filter

Exercise Description

Write a function `filter_above_threshold(d, threshold)` that takes a dictionary `d` of item prices and a threshold value. It should return a new dictionary with only items priced above the threshold.

Your solution

```
def filter_above_threshold(d, threshold):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1: Filter prices above 2
prices = {"apple": 2, "banana": 1, "cherry": 3, "date": 5}
result = filter_above_threshold(prices, 2)
assert result == {'cherry': 3, 'date': 5}
```

Test your solution

```python
# Test case 2: Filter with high threshold
prices = {"apple": 2, "banana": 1, "cherry": 3}
result = filter_above_threshold(prices, 5)
assert result == {}
```

Test your solution

```python
# Test case 3: Filter with low threshold
prices = {"apple": 2, "banana": 1, "cherry": 3}
result = filter_above_threshold(prices, 0)
assert result == {"apple": 2, "banana": 1, "cherry": 3}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Unique Elements with Nested Sets

Exercise Description

You have a list of sets `data = [{1, 2, 3}, {2, 3, 4}, {4, 5}]`. Write a function `extract_unique_elements(data)` that returns a set of all unique elements across these sets

Your solution

```python
def extract_unique_elements(data):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1: Extract unique elements from nested sets
data = [{1, 2, 3}, {2, 3, 4}, {4, 5}]
result = extract_unique_elements(data)
assert result == {1, 2, 3, 4, 5}
```

Test your solution

```python
# Test case 2: Empty list of sets
data = []
result = extract_unique_elements(data)
assert result == set()
```

Test your solution

```
# Test case 3: Single set
data = [{1, 2, 3}]
result = extract_unique_elements(data)
assert result == {1, 2, 3}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Find the Symmetric Difference of Two Lists

Exercise Description

Write a function `symmetric_difference(lst1, lst2)` that returns the symmetric difference of two lists as a set. Elements should only appear if they are in one list but not both.

Your solution

```python
def symmetric_difference(lst1, lst2):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1: Symmetric difference of two lists
lst1 = [1, 2, 3, 4]
lst2 = [3, 4, 5, 6]
result = symmetric_difference(lst1, lst2)
assert result == {1, 2, 5, 6}
```

Test your solution

```python
# Test case 2: No common elements
lst1 = [1, 2]
lst2 = [3, 4]
result = symmetric_difference(lst1, lst2)
assert result == {1, 2, 3, 4}
```

Test your solution

```python
# Test case 3: All elements are common
lst1 = [1, 2, 3]
lst2 = [1, 2, 3]
result = symmetric_difference(lst1, lst2)
assert result == set()
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Dictionary Difference

Exercise Description

Write a function `dict_difference(dict1, dict2)` that takes two dictionaries and returns a new dictionary with keys that are only in `dict1` or `dict2`, but not both.

Your solution

```
def dict_difference(dict1, dict2):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1: Dictionary difference
dict1 = {"a": 1, "b": 2, "c": 3}
dict2 = {"b": 2, "d": 4, "e": 5}
```

```
result = dict_difference(dict1, dict2)
assert result == {'a': 1, 'c': 3, 'd': 4, 'e': 5}
```

Test your solution

```
# Test case 2: No common keys
dict1 = {"a": 1, "b": 2}
dict2 = {"c": 3, "d": 4}
result = dict_difference(dict1, dict2)
assert result == {"a": 1, "b": 2, "c": 3, "d": 4}
```

Test your solution

```
# Test case 3: All keys are common
dict1 = {"a": 1, "b": 2}
dict2 = {"a": 1, "b": 2}
result = dict_difference(dict1, dict2)
assert result == {}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Tuple Indexing and Modification

Exercise Description

Given a tuple `data = (1, [2, 3], 4)`, write code to change the list element to `[2, 5]`. Note that tuples are immutable, but you can modify mutable elements within them. Store the result in a variable called `result`.

Your solution

```
data = (1, [2, 3], 4)
# Write your solution here to modify the list within the tuple
# Store the modified tuple in the variable 'result'
```

Test your solution

```
# Test case 1: Check if the list within tuple was modified correctly
assert result == (1, [2, 5], 4)
```

Test your solution

```
# Test case 2: Check that the list element was changed
assert result[1][1] == 5
```

Test your solution

```python
# Test case 3: Check that other elements remain unchanged
assert result[0] == 1
assert result[2] == 4
assert result[1][0] == 2
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Deep Merging of Dictionaries

Exercise Description

Write a function `deep_merge(dict1, dict2)` that merges two dictionaries. If there are nested dictionaries, it should merge them recursively. Otherwise, `dict2` values should overwrite `dict1`.

Your solution

```python
def deep_merge(dict1, dict2):
    # Write your solution here
```

```
    pass
```

Test your solution

```
# Test case 1: Deep merge with nested dictionaries
dict1 = {"a": 1, "b": {"x": 10, "y": 20}}
dict2 = {"b": {"y": 30, "z": 40}, "c": 3}
result = deep_merge(dict1, dict2)
assert result == {'a': 1, 'b': {'x': 10, 'y': 30, 'z': 40}, 'c': 3}
```

Test your solution

```
# Test case 2: Simple merge without nested dictionaries
dict1 = {"a": 1, "b": 2}
dict2 = {"b": 3, "c": 4}
result = deep_merge(dict1, dict2)
assert result == {"a": 1, "b": 3, "c": 4}
```

Test your solution

```
# Test case 3: Empty dictionaries
dict1 = {}
dict2 = {"a": 1}
result = deep_merge(dict1, dict2)
assert result == {"a": 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: Inverse Mapping of a Dictionary

Exercise Description

Write a function `invert_dict(d)` that takes a dictionary `d` where values are lists, and returns a new dictionary where each element in the list is a key and maps to the original dictionary key.

Your solution

```
def invert_dict(d):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1: Invert dictionary with list values
data = {"fruits": ["apple", "banana"], "vegetables": ["carrot", "spinach"]}
result = invert_dict(data)
```

```
expected = {'apple': 'fruits', 'banana': 'fruits', 'carrot': 'vegetables', 'spinach':
assert result == expected
```

Test your solution

```
# Test case 2: Empty dictionary
data = {}
result = invert_dict(data)
assert result == {}
```

Test your solution

```
# Test case 3: Dictionary with empty lists
data = {"category1": [], "category2": ["item1"]}
result = invert_dict(data)
assert result == {"item1": "category2"}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Flatten Nested Tuples

Exercise Description

Given a tuple `data = (1, (2, (3, 4)), 5)`, write a function `flatten_tuple(t)` that flattens it into a single tuple `(1, 2, 3, 4, 5)`.

Your solution

```
def flatten_tuple(t):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1: Flatten nested tuples
data = (1, (2, (3, 4)), 5)
result = flatten_tuple(data)
assert result == (1, 2, 3, 4, 5)
```

Test your solution

```
# Test case 2: Simple tuple without nesting
data = (1, 2, 3)
result = flatten_tuple(data)
assert result == (1, 2, 3)
```

Test your solution

```python
# Test case 3: Empty tuple
data = ()
result = flatten_tuple(data)
assert result == ()
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Cartesian Product of Sets

Exercise Description

Write a function `cartesian_product(set1, set2)` that returns the Cartesian product of two sets as a set of tuples.

Your solution

```python
def cartesian_product(set1, set2):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1: Cartesian product of two sets
set1 = {1, 2}
set2 = {"a", "b"}
result = cartesian_product(set1, set2)
expected = {(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')}
assert result == expected
```

Test your solution

```
# Test case 2: Empty sets
set1 = set()
set2 = {1, 2}
result = cartesian_product(set1, set2)
assert result == set()
```

Test your solution

```
# Test case 3: Single element sets
set1 = {1}
set2 = {"a"}
result = cartesian_product(set1, set2)
assert result == {(1, "a")}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 11: Difference of Multiple Sets

Exercise Description

Write a function `multiple_set_difference(*sets)` that takes multiple sets and returns a set containing elements only found in the first set and not in any others.

Your solution

```
def multiple_set_difference(*sets):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1: Multiple set difference
set1 = {1, 2, 3, 4}
set2 = {2, 3}
set3 = {4}
result = multiple_set_difference(set1, set2, set3)
assert result == {1}
```

Test your solution

```
# Test case 2: No sets provided
result = multiple_set_difference()
assert result == set()
```

Test your solution

```
# Test case 3: Single set
set1 = {1, 2, 3}
result = multiple_set_difference(set1)
assert result == {1, 2, 3}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 12: Grouping Elements by Category

Exercise Description

Write a function `group_by_value(d)` that takes a dictionary `d` where values are categories and returns a new dictionary where each key is a category and the value is a list of all keys from the original dictionary that map to this category.

Your solution

```
def group_by_value(d):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1: Group by category
data = {"apple": "fruit", "carrot": "vegetable", "banana": "fruit", "spinach": "veget
result = group_by_value(data)
expected = {'fruit': ['apple', 'banana'], 'vegetable': ['carrot', 'spinach']}
assert result == expected
```

Test your solution

```
# Test case 2: Empty dictionary
data = {}
result = group_by_value(data)
assert result == {}
```

Test your solution

```python
# Test case 3: Single category
data = {"apple": "fruit", "banana": "fruit"}
result = group_by_value(data)
assert result == {"fruit": ["apple", "banana"]}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 13: Find Unique Pairs with a Condition

Exercise Description

Write a function `find_unique_pairs(lst, target_sum)` that takes a list of integers and a target sum. The function should return a list of unique pairs (as tuples) that add up to the target sum.

Your solution

```python
def find_unique_pairs(lst, target_sum):
    # Write your solution here
```

```python
    pass
```

Test your solution

```python
# Test case 1: Find pairs that sum to target
lst = [1, 3, 2, 2, 4, 5, 3]
target_sum = 5
result = find_unique_pairs(lst, target_sum)
assert sorted(result) == sorted([(1, 4), (2, 3)])
```

Test your solution

```python
# Test case 2: No pairs found
lst = [1, 2, 3]
target_sum = 10
result = find_unique_pairs(lst, target_sum)
assert result == []
```

Test your solution

```python
# Test case 3: Multiple pairs
lst = [1, 2, 3, 4, 5, 6]
target_sum = 7
result = find_unique_pairs(lst, target_sum)
assert result == [(1, 6), (2, 5), (3, 4)]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 14: Merge Dictionaries with Conflicting Keys

Exercise Description

Write a function `merge_dicts(d1, d2)` that merges two dictionaries. If both dictionaries have the same key, their values should be added; otherwise, keep the existing value.

Your solution

```
def merge_dicts(d1, d2):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1: Merge dictionaries with conflicting keys
d1 = {"a": 5, "b": 3}
d2 = {"a": 2, "c": 7}
result = merge_dicts(d1, d2)
assert result == {'a': 7, 'b': 3, 'c': 7}
```

Test your solution

```python
# Test case 2: No conflicting keys
d1 = {"a": 1, "b": 2}
d2 = {"c": 3, "d": 4}
result = merge_dicts(d1, d2)
assert result == {"a": 1, "b": 2, "c": 3, "d": 4}
```

Test your solution

```python
# Test case 3: Empty dictionaries
d1 = {}
d2 = {"a": 1}
result = merge_dicts(d1, d2)
assert result == {"a": 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 15: Filter Dictionary by Key Prefix

Exercise Description

Write a function `filter_by_prefix(d, prefix)` that takes a dictionary and a prefix string. It should return a new dictionary containing only items whose keys start with the given prefix.

Your solution

```
def filter_by_prefix(d, prefix):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1: Filter by prefix "ap"
data = {"apple": 2, "banana": 3, "apricot": 4, "berry": 5}
result = filter_by_prefix(data, "ap")
assert result == {'apple': 2, 'apricot': 4}
```

Test your solution

```python
# Test case 2: No matches
data = {"apple": 2, "banana": 3}
result = filter_by_prefix(data, "z")
assert result == {}
```

Test your solution

```python
# Test case 3: Empty prefix (should return all)
data = {"apple": 2, "banana": 3}
result = filter_by_prefix(data, "")
assert result == {"apple": 2, "banana": 3}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 16: Find Unique Elements in a List of Tuples

Exercise Description

Write a function `unique_elements(tuples_list)` that takes a list of tuples and returns a set of unique elements found in all tuples.

Your solution

```
def unique_elements(tuples_list):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1: Find unique elements in list of tuples
tuples_list = [(1, 2), (2, 3), (4, 1)]
result = unique_elements(tuples_list)
assert result == {1, 2, 3, 4}
```

Test your solution

```
# Test case 2: Empty list
tuples_list = []
result = unique_elements(tuples_list)
assert result == set()
```

Test your solution

```
# Test case 3: Single tuple
tuples_list = [(1, 2, 3)]
```

```python
result = unique_elements(tuples_list)
assert result == {1, 2, 3}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 17: Count Frequency of Tuple Pairs

Exercise Description

Write a function `count_tuple_pairs(pairs)` that takes a list of tuples and returns a dictionary with each tuple as a key and its frequency as the value.

Your solution

```python
def count_tuple_pairs(pairs):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1: Count frequency of tuple pairs
pairs = [(1, 2), (2, 3), (1, 2), (3, 4)]
result = count_tuple_pairs(pairs)
assert result == {(1, 2): 2, (2, 3): 1, (3, 4): 1}
```

Test your solution

```python
# Test case 2: Empty list
pairs = []
result = count_tuple_pairs(pairs)
assert result == {}
```

Test your solution

```python
# Test case 3: All unique pairs
pairs = [(1, 2), (3, 4), (5, 6)]
result = count_tuple_pairs(pairs)
assert result == {(1, 2): 1, (3, 4): 1, (5, 6): 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 18: Reverse Lookup

Exercise Description

Write a function `reverse_lookup(d, value)` that returns all keys from dictionary `d` that map to the given `value`.

Your solution

```
def reverse_lookup(d, value):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
d = {'a': 1, 'b': 2, 'c': 1}
result = reverse_lookup(d, 1)
print(result)  # Expected: ['a', 'c']
```

Test your solution

```
# Test case 2
d = {'x': 'hello', 'y': 'world', 'z': 'hello'}
result = reverse_lookup(d, 'hello')
print(result)  # Expected: ['x', 'z']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 19: Count Tuple Elements

Exercise Description

Write a function `count_elements(tup)` that returns a dictionary with counts of each element in the tuple `tup`.

Your solution

```
def count_elements(tup):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
tup = (1, 2, 2, 3, 1)
```

```python
result = count_elements(tup)
print(result)  # Expected: {1: 2, 2: 2, 3: 1}
```

Test your solution

```python
# Test case 2
tup = ('a', 'b', 'a', 'c', 'b', 'a')
result = count_elements(tup)
print(result)  # Expected: {'a': 3, 'b': 2, 'c': 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 20: Set Intersection

Exercise Description

Implement a function `intersect(s1, s2)` that returns the intersection of two sets `s1` and `s2`.

Your solution

```python
def intersect(s1, s2):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1
s1 = {1, 2, 3}
s2 = {2, 3, 4}
result = intersect(s1, s2)
print(result)  # Expected: {2, 3}
```

Test your solution

```python
# Test case 2
s1 = {'a', 'b', 'c'}
s2 = {'b', 'c', 'd'}
result = intersect(s1, s2)
print(result)  # Expected: {'b', 'c'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 21: Remove Duplicates

Exercise Description

Write a function `remove_duplicates(lst)` that returns a list without any duplicates using a set.

Your solution

```
def remove_duplicates(lst):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
lst = [1, 2, 2, 3, 4, 4, 5]
result = remove_duplicates(lst)
print(result)  # Expected: [1, 2, 3, 4, 5]
```

Test your solution

```python
# Test case 2
lst = ['a', 'b', 'a', 'c', 'b']
result = remove_duplicates(lst)
print(result)  # Expected: ['a', 'b', 'c'] (order may vary)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 22: Tuple to Dictionary

Exercise Description

Create a function `tuple_to_dict(tup)` that turns a tuple of `(key, value)` pairs into a dictionary.

Your solution

```python
def tuple_to_dict(tup):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
tup = (('a', 1), ('b', 2))
result = tuple_to_dict(tup)
print(result)  # Expected: {'a': 1, 'b': 2}
```

Test your solution

```
# Test case 2
tup = (('x', 10), ('y', 20), ('z', 30))
result = tuple_to_dict(tup)
print(result)  # Expected: {'x': 10, 'y': 20, 'z': 30}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 23: Value Switch in Dictionary

Exercise Description

Write a function `switch_values(d)` that switches the keys and values in a dictionary. Assume all values are unique.

Your solution

```
def switch_values(d):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
d = {'a': 1, 'b': 2}
result = switch_values(d)
print(result)  # Expected: {1: 'a', 2: 'b'}
```

Test your solution

```
# Test case 2
d = {'name': 'John', 'age': 25}
result = switch_values(d)
print(result)  # Expected: {'John': 'name', 25: 'age'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 24: Dictionary of Squares

Exercise Description

Create a function `squares(n)` that returns a dictionary of numbers from 1 to `n` where the keys are numbers and the values are their squares.

Your solution

```python
def squares(n):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1
result = squares(5)
print(result)  # Expected: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Test your solution

```python
# Test case 2
result = squares(3)
```

```python
print(result)  # Expected: {1: 1, 2: 4, 3: 9}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 25: Set Difference

Exercise Description

Write a function `difference(s1, s2)` that returns a set of elements that are only in the first set `s1` but not in the second set `s2`.

Your solution

```python
def difference(s1, s2):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
result = difference(s1, s2)
print(result)  # Expected: {1, 2}
```

Test your solution

```python
# Test case 2
s1 = {'a', 'b', 'c'}
s2 = {'b', 'd'}
result = difference(s1, s2)
print(result)  # Expected: {'a', 'c'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 26: Count Vowels

Exercise Description

Create a function `count_vowels(s)` that counts and returns a dictionary with vowels as keys and their counts in the string `s` as values.

Your solution

```
def count_vowels(s):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
result = count_vowels('hello world')
print(result)  # Expected: {'e': 1, 'o': 2}
```

Test your solution

```
# Test case 2
result = count_vowels('programming')
print(result)  # Expected: {'o': 1, 'a': 1, 'i': 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 27: Find Max Value

Exercise Description

Write a function `find_max(d)` that returns the key with the maximum value in the dictionary `d`.

Your solution

```
def find_max(d):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
d = {'a': 5, 'b': 12, 'c': 3}
result = find_max(d)
print(result)  # Expected: 'b'
```

Test your solution

```
# Test case 2
d = {'x': 100, 'y': 50, 'z': 200}
result = find_max(d)
print(result)  # Expected: 'z'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 28: Unique Characters

Exercise Description

Create a function `unique_chars(s)` that returns a set of unique characters in the string `s`.

Your solution

```
def unique_chars(s):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
result = unique_chars('hello')
print(result)  # Expected: {'e', 'h', 'l', 'o'}
```

Test your solution

```
# Test case 2
result = unique_chars('programming')
print(result)  # Expected: {'p', 'r', 'o', 'g', 'a', 'm', 'i', 'n'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 29: Key Multiplication

Exercise Description

Write a function `multiply_keys(d, factor)` that returns a new dictionary where each key is multiplied by a `factor`.

Your solution

```
def multiply_keys(d, factor):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
d = {1: 100, 2: 200}
result = multiply_keys(d, 2)
print(result)  # Expected: {2: 100, 4: 200}
```

Test your solution

```
# Test case 2
d = {3: 'a', 5: 'b'}
result = multiply_keys(d, 10)
print(result)  # Expected: {30: 'a', 50: 'b'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 30: Set Complement

Exercise Description

Implement a function `complement(full_set, sub_set)` that returns the set of elements in `full_set` that are not in `sub_set`.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1
full_set = {1, 2, 3, 4, 5}
sub_set = {2, 4}
result = complement(full_set, sub_set)
print(result)  # Expected: {1, 3, 5}
```

Test your solution

```
# Test case 2
```